Negotiate, Then Solve

Solver-Grounded Negotiation for Conflict-Aware University Timetabling

Mid-Submission Report

Team Farali Chevdo

Achyutananda Sahoo	2025202028	Rakesh C	2026901015
Satyajit Priyadarshi	2025201055	Shubhadeep Mandal	2025201056

Code: https://github.com/Seronic2001/negotiate-then-solve

Summary. The full system in the proposal is built and runs end to end, from a plain-language request to a human-approved timetable, in a web application. The **main experiment is complete**: solver-grounded negotiation reaches the oracle's outcome in **89.4%** of runs against **75.6%** for generate-and-filter B4 (p=4.6×10^{-6}) and **83.3%** for unverified B3 (p=0.013), with an oracle gap of 0.04 against 0.63 and 0.44. Every capability has a component benchmark with a baseline, and a fine-tuned **tool-calling coordinator agent** reaches 100% on our 19-family suite (Gemini Flash-Lite: 85%). Remaining: a same-model control, judge validation, a test of the fairness ledger (H4 not yet supported) and the human pilot.

1 Recap: Problem, Research Question and Hypotheses

University timetables change weekly after publication; change requests arrive as free text, and when two requests cannot both be met there is no principled way to decide who concedes. We study *solver-grounded negotiation*: when a request is infeasible, minimal unsatisfiable subsets (MUS) locate the conflict, minimal correction sets (MCS) define the possible concessions, CP-SAT re-solves each into a concrete timetable, and an LLM negotiator may offer *only* these verified options, ranked by priority tiers and a concession ledger. A coordinator approves every change.

RQ. Can solver-derived conflict sets constrain an LLM negotiator so that it resolves infeasible timetable requests with fewer negotiation rounds and lower disruption than LLM-invented options, even when those options are filtered by the same solver before they are offered?

H1 invented options cost rounds or regeneration calls; **H2** ours resolves at least as often as B4 with fewer rounds, less disruption and a smaller oracle gap; **H3** grounded explanations are more faithful and leak no private reasons; **H4** the ledger spreads concession burden more evenly than flat weights.

2 Progress Since the Proposal

Component	What is built	Status
Pipeline L1–L6	Receive (portal, email, messaging), parse, policy check (RAG), CP-SAT repair, negotiate (MUS, MCS, re-solve, ranked grounded options), approve and publish; role-scoped web app	done
Request reader	Fine-tuned reader into a typed *intent language* (17 kinds incl. limits, relations, choices); deterministic checks repair or question a reading	done; v12 in evaluation
Coordinator agent	Tool-calling agent (30 tools) that handles a whole request over many turns; SFT + DPO	v4 evaluated; v6 trained
Evaluation	Negotiation runner with simulators and oracle; benchmarks per capability; LLM cache for exact replay; about 600 tests	done
Human study	Participant codes, two raters per item, rating UI	pending ethics

Table 1: Implementation status.

Deviations from the proposal. (i) The local model is **Qwen3.5-4B** + LoRA, not 2B: the 4B adapter had higher action accuracy (99.3% vs 96.5%, Table 5). (ii) Requests are read into **intents** that a deterministic compiler resolves into constraints, so each reading can be checked against the message

and timetable. (iii) A tool-calling **coordinator agent** (Section 5.4) handles what the fixed pipeline could not express (swaps, make-up classes, multi-part requests).

3 Datasets

All data is synthetic or team-authored, with ground truth known by construction.

Dataset	Construction	Size
Department instance	Synthetic CSE department: 30 faculty, 10 groups, 20 rooms, 5 days × 8 slots	202 sessions
A. Negotiation	Planted conflicts of 7 kinds; hidden stakeholder flexibility; solver oracle	60 scenarios
B. Requests / intents	Requests from known targets, paraphrased, ambiguous, adversarial; intent data (v12) with 24 held-out suites	600; v12 4,363 train
C. Policy (RAG)	Handbook (32 rules) + 4 circulars; queries with gold rule IDs	106 + 40 paraphrased
D. Replies, safety	Stakeholder replies with the correct tool call; injections, authority violations	100; 48
Agent	19 request families; verified trajectories for training	38 per seed

Table 2: Datasets.

4 Main Experiment: Solver-Grounded Negotiation (H1–H4)

Setup. Ours, B3 and B4 ran 60 scenarios × 3 seeds (180 paired runs). Ours uses the fine-tuned Qwen3.5-4B (local); the baselines use the much larger **Gemini 3.5 Flash-Lite**, so they are not handicapped on language. Four simulator families play the stakeholders. A run is *correct* when its final status matches the oracle's.

System	n	Correct outcome	Agreement	Rounds	Oracle gap	Invalid options
B1 LLM-only	20	0.0%	0.0%	–	–	80% of TTs
B2 Solver-only	60	31.7%	0.0%	–	0.131	–
B3 Unverified	180	83.3%	66.7%	1.47	0.443	51.0%
B4 Filtered	180	75.6%	51.1%	1.48	0.628	5.7 filtered/sc.
Ours	180	**89.4%**	**78.9%**	**1.38**	**0.041**	**0**

Table 3: Main results (oracle gap: objective difference to the oracle's repair, lower is better). McNemar on correct outcome: Ours vs B4 p=4.6×10^{-6}, vs B3 p=0.013; differences in rounds are not significant.

Hyp.	Evidence so far	Verdict
H1	51% of B3's options are infeasible; B4 needs 3.97 generation calls per scenario yet has the lowest agreement. Rounds not significantly higher.	**supported** (rounds n.s.)
H2	Correct 89.4% vs 75.6%, oracle gap 0.04 vs 0.63; rounds equal within CI.	**supported** (except rounds)
H3	Grounded vs free-text (A2): 0 vs 12 private-reason leaks; claim faithfulness 0.98, not yet human-validated.	**partly**
H4	Ledger vs flat weights: Gini 0.241 vs 0.224; one conflict per scenario gives the ledger little to act on.	**not yet** (M6)

Table 4: Hypotheses: status at mid-submission.

Ablations. Flat weights (A1), free-text explanations (A2) and a regex reply handler (A3) leave resolution unchanged (90.0%); they differ in what the stakeholder sees and in burden spread. Imposing the best repair without asking (B2) is correct in only 31.7% of scenarios: negotiation is needed, not only a solver.

5 Component Results

5.1 Fine-tuning: reading requests

Reader (151 test requests)	Compile exact match	Atom F1	Action accuracy
Gemini 3.5 Flash-Lite (prompted)	92.4%	0.977	99.3%
Qwen3.5-2B + LoRA	**97.8%**	**0.996**	96.5%
Qwen3.5-4B + LoRA	**97.8%**	**0.996**	**99.3%**

Table 5: Constraint compilation: both adapters beat the prompted large model.

The newer **intent reader** also handles compositional requests: limits ("at most two days a week"), relations ("the practical after the lectures") and choices ("Tuesday, failing that Thursday"). Scores are outcome match / silent wrong (%).

Reader	test	test-typo	compositional	relations
Rule parser (baseline)	51.6 / 21.1	38.6 / 24.9	50.0 / 28.6	0.0 / 5.0
v11	95.4 / 2.4	94.2 / 3.1	78.6 / 14.3	45.0 / 0
v12	**98.1 / 1.9**	**97.6 / 1.9**	**85.7 / 0.0**	**77.5 / 0**

Table 6: Intent readers on the frozen test split and held-out suites.

5.2 RAG: policy check with citations

Retrieval over the handbook and circulars reaches recall@5 of 0.93 (hybrid); lexical retrieval is best on raw queries and dense on paraphrases.

Policy verdict	Allow/deny accuracy	Deny recall (cited)	Make-up obligation recall
Gemini 3.5 Flash-Lite	98%	0.80	0.21
Qwen3.5-4B + LoRA	99%	0.90	**1.00**

Table 7: The fine-tuned reader learns the make-up-class rule the prompted model misses.

5.3 Tool calling and safety: stakeholder replies

Reply handler (100 replies)	Tool accuracy	Argument accuracy	Unsafe actions
Qwen3.5-4B base (prompted)	65%	100%	0
Gemini 3.5 Flash-Lite	**100%**	85.5%	0
Qwen3.5-4B + LoRA	**100%**	**100%**	0

Table 8: Reply-to-tool-call accuracy. On the 48-case safety set every case is handled safely; no case changed the published timetable.

5.4 Agentic: the tool-calling coordinator

The agent works a whole request through tools (look up, move, swap, cancel, make-up class, check policy, ask, hand off).

Agent	Default (19 families)	Held-out	Stress	Compositional
Qwen3.5-4B base (prompted)	66.7%	–	–	–
Gemini 3.5 Flash-Lite	85.1%	–	90.0%	–
nts-agent-v3 (SFT + DPO)	98.2%	95.0%	59.4%	–
nts-agent-v4 (SFT)	**100%**	**100%**	**100%**	42.9%

Table 9: Agent pass rate. v6, trained on compositional requests, is evaluated next (M1).

6 Failure Analysis

- **Where Ours fails (19 of 180 runs).** Every miss is a safe escalation where the oracle reached agreement: the stakeholder did not reply in time (11) or the acceptable option was not among those offered (8). No run offered an infeasible timetable.
- **B4.** The solver rejects most invented options; the survivors are fewer and worse, which explains why B4 is *less* correct than unverified B3.
- **Reader (v12).** Most held-out misses come from two patterns: a course name lost in "any day apart from Friday works for my Networks classes", and the second course copied into the first in two-request messages.

7 Challenges and Mitigation

Challenge	Mitigation
One shared GPU laptop; outages; Gemini free-tier limits	One adapter per run, all answers cached so re-scoring is free, paced API keys
Firmness label drift: "hard" vs "soft" labelled inconsistently across data versions	One labelling rule in code with checks (agreement 86.6%→96.7%)
Kaggle 12 h limit cut agent-v6 DPO short	Second DPO-only session (launched)
Main comparison mixes models (4B vs Gemini)	Swap models on a 20-scenario subset (M2)
Faculty pilot needs ethics clearance	Peer study with the same protocol if late

Table 10: Challenges and mitigations.

8 Plan to the Final Submission (15 days)

Days	Work	Deliverable
1–2	**M1** Finish reader scoring; evaluate agent-v6	Final reader, agent tables
2–5	**M2** Same-model control. **M3** v13 reader fixes	Model-bias check
4–7	**M4** Judge validation against human labels (Cohen's κ)	Validated H3
5–9	**M5** Human pilot or peer study	Pilot results
6–10	**M6** Sequential-request scenarios to test the ledger	H4 result
9–15	**M7** Failure analysis, final report, cleaned repository, demo	Final submission

Table 11: Timeline. If short of time we drop v13 first, then the pilot; never M2.